

School of Electrical & Information Engineering, University of the
Witwatersrand

Reverse 2048 Solver: Evaluating Greedy vs Expectimax Algorithm

Software Development (ELEN 2020) Project

Kallisha Naidoo (2803488), Akiria Chetty (2801838)
5-5-2025

Abstract

Reverse 2048 is a puzzle game of chance with the main objective of merging larger tiles to achieve the smallest tile through a series of strategic moves. This project aims to use two AI algorithms, Greedy and Expectimax, to solve the game in various configurations to observe which algorithm is best suited for this game. Greedy AI prioritises moves that give an immediate reward, while Expectimax uses a heuristic approach to analyse and predict moves using probability. Experimental results show that Expectimax is more efficient in solving the game, proving its suitability for Reverse 2048 and its variants.

1. Introduction

Reverse 2048 is a variation of the classic 2048 game that challenges players to divide tiles by merging them until the smallest tile is achieved. This single-player game is usually played on a 4x4 board, consisting of tiles that could be empty or contain a value (which is a multiple of 2). Instead of merging tiles to reach an end goal of 2048, the objective of Reverse 2048 is to merge and divide tiles to the value of 2. The main goal of this puzzle game is to minimise tiles instead of maximising them. To merge tiles, the player slides the tiles in four possible directions: up, down, left and right. When two tiles (consisting of the same value) collide, they merge into one tile that is half the value of the original tile. Finally, the game ends when no more valid moves can be made or the smallest number (2) has been achieved.

This project was implemented in C++ and can read input files and stimulate gameplay for different game variations. The game variations include: Reverse 512, 256 and 128. The main objective of this project is to implement two algorithms that play the reverse 2048 game, and the first one to achieve a tile value of 2 is declared the winner. The challenge lies in making optimal moves to minimise tile values to reach the smallest tile value of 2 before the board becomes saturated and no further moves are possible. The first algorithm demonstrates a simplistic approach to solving the game, while the second algorithm demonstrates a more advanced, heuristic-based AI algorithm that aims to reduce tile values efficiently. Since algorithm 2 is a more complex algorithm, it is expected to perform better in all configurations of the reverse game.

Integrating an AI algorithm requires knowledge of specific concepts; therefore, without this knowledge, implementation can be difficult. Our C++ familiarity influenced the development pace as unfamiliar concepts required additional learning.

1.1 Background and Equations

The use of artificial intelligence in puzzle games has become popular for evaluating decision-making strategies (Bach, 2022). AI algorithms are beneficial when used to solve puzzles, as they can analyse and generate various potential solutions in a short time interval (Bach, 2022). These algorithms can solve puzzles more quickly than humans, thus making AI a powerful tool (Bach, 2022).

AI algorithms used in Reverse 2048 include Greedy and Expectimax algorithms. A Greedy algorithm is a simple approach to solving the game by prioritising moves based on instant rewards (Design Gurus Team, 2020). However, this algorithm may struggle with long-term planning and may perform poorly in complex configurations (Design Gurus Team, 2020). An Expectimax algorithm uses a depth-limited search to gain foresight for the best possible move (Design Gurus Team, 2020). This algorithm evaluates all possible new tiles and the tile locations of the next generation (Chowdhury & Dhamodaran, 2014).

Comparison of the two algorithms will be based on the win rate of each algorithm. The performance is evaluated over a fixed number of test runs, and the win rate is calculated as follows:

$$\text{Win Rate (\%)} = \frac{(\text{Number of wins} / \text{Total Games played}) \times 100}{(1)}$$

2. Design

2.1 Game mechanics:

The game is played on a board that varies in size and starts with two random tiles generated in a random position. Both algorithms operate under the same initial conditions and make moves independently without any external user input. The Algorithms can move the tile in all four directions: up, down, left and right. After each move, a new tile is generated in a space and the state of the board changes. If two tiles have the same numerical value and they merge during a move, the value of the tile is halved. The game ends when one of three end conditions is met.

2.2 Initial state of the game:

There are multiple variants and board sizes available. Therefore, a function must be created to receive input from a text file containing the type of game variant and its board size. A random number generator is needed to generate the new tiles based on the variant.

2.3 End conditions:

A check will be implemented to see if any tile values are 2, if 1000 moves have been made, or if the space is full and no valid moves can be made. The appropriate response will be output.

2.4 File handling:

The program receives input from a text file that contains the board size and Reverse 2048 variant. It opens the input file, and creates an output file. As the program reads the input, the Greedy and Expectimax algorithms execute and write their corresponding results to the output file. Once the input file has been read, the program closes all files.

2.5 Random Number Generator:

A class is used to manage the random number generator for tile placement. The class searches the board for unoccupied spaces and returns a random position. A random tile appears in that position to display. If there are no empty tiles, it returns an invalid position and no tile will be generated.

2.6 Algorithm 1: Greedy

Aim: The algorithm selects the move that produces the best result at the moment, that being the number of tiles that are merged at that moment.

Assumptions: This algorithm lacks foresight as it prioritises immediate results. The algorithm is expected to perform poorly against Algorithm 2, which is more strategic.

Implementation: The Greedy algorithm stores the original board in four variables that detail one of the four directions. A direction function is called to simulate the move. During simulation, if two tiles can merge, the value halves and the board shifts again. This function changes the board and counts the number of merges made in each direction. An array is used to store the different merge counts, and the direction with the highest merge count is chosen.

If the merge counts are zero, the algorithm chooses a random direction to move in by looping through a vector containing the directions. If there are no valid moves or a tile has the value of 2, the algorithm cannot make a move.

2.7 Algorithm 2: Expectimax

Aim: This algorithm will use probability to solve the game. The value of the probability will be calculated through simulating and anticipating moves using functions.

Assumptions: This algorithm uses a heuristic approach to predict moves to achieve the best result in the long term. Therefore, it will outperform Algorithm 1.

Implementation: This algorithm uses two vector functions. "Expectimax" and "Evaluate Board".
Expectimax: If it is the algorithm's turn, it receives a depth value as a parameter from the main program. If that value is zero, there are no further moves to look into, so it calls the "Evaluate Board" function. A best score is initialised to a low value so that it is easy to beat. A vector is created to store all four directions, to call the direction functions, and to simulate moves. If the merge counts have a value and the board changes during simulation, the "Expectimax" function is called again and the value of the best score changes. If it is not the algorithm's turn, the function simulates random placement of tiles over empty spaces. Assume that all possible tiles appear and change the expected score using probability. If there are no empty spaces, the "Evaluate Board" function is called.

Evaluate Board: This function loops through the board to count empty spaces and finds the tile with the lowest value. It checks the location of the tile and calculates a score to return to the Expectimax function.

3. Results

To test the algorithms, the program was run multiple times for data collection. This data was then used to calculate the win rate for each algorithm. The win rates for algorithm 1 were compared to the win rates for algorithm 2, to see which algorithm performed better and is well-suited for the Reverse 2048 game.

3.1 Test 1

Testing was conducted using the input file “reverse_2048.txt”. The program was run 20 times for data collection. The win rates across the different reverse game configurations were calculated using this data:

Table 1: The Number of Wins for each Algorithm (out of 20) corresponding to the reverse game.

Reverse Game (Board size)	Win Count	
	Alg_1	Alg_2
512 (3x3)	0	0
128 (4x4)	1	19
512 (5x5)	0	20
256 (4x4)	3	17

Table 2: The Win rates for each Algorithm corresponding to the Reverse game

Reverse Game (Board Size)	Alg_1 Win Rate (%)	Alg_2 Win Rate (%)
512 (3x3)	0	0
128 (4x4)	5	95
512 (5x5)	0	100
256 (4x4)	15	85

A sample calculation for the win rate, using equation 1, is shown below:

Reverse Game 256 (4x4)

$$\text{Win Rate } Alg_1 (\%) = (3/20) \times 100$$

$$\text{Win Rate } Alg_1 = 15\%$$

$$\text{Win Rate } Alg_2 (\%) = (17/20) \times 100$$

$$\text{Win Rate } Alg_2 = 85\%$$

3.2 Test 2

A second test was done, testing all of the possible game configurations. The input file was changed to accommodate all reverse game configurations. The program was run 10 times for data collection. The win rates per reverse game variant were then calculated using the same procedure as Test 1. The results are depicted in the graphs that follow:

Reverse 512:

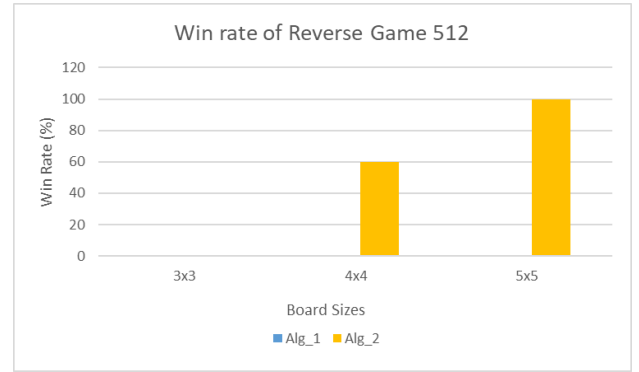


Figure 1: Bar Graph comparing Win Rates for Reverse 512

Reverse 256:

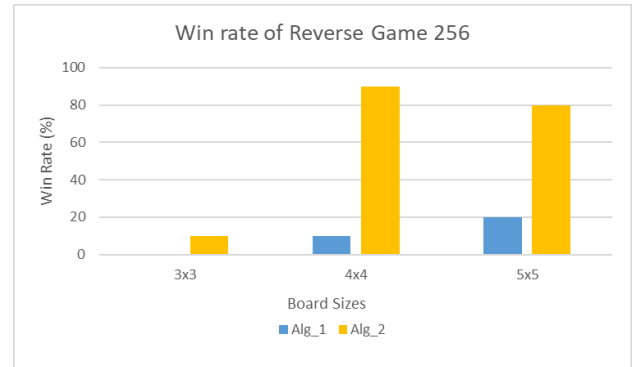


Figure 2: Bar Graph comparing Win Rates for Reverse 256

Reverse 128:

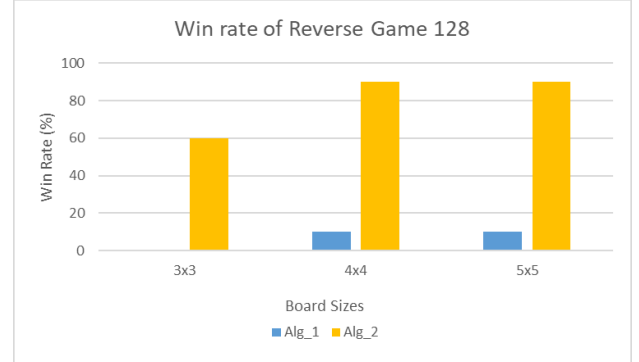


Figure 3: Bar Graph comparing Win Rates for Reverse 128

4. Discussion

4.1 Test 1

Algorithm 2 outperformed Algorithm 1 in nearly all reverse game configurations except for the 512 (3x3) configuration, where neither algorithm won the game. This outcome can be attributed to the spatial constraints of the 3x3 board, which led to early board saturation and limited movement opportunities.

This emphasises the difficulty of the Reverse 512 (3x3) game and suggests that smaller boards significantly increase the challenge for both algorithms, regardless of their approach.

The strongest performance from algorithm 2 occurred in the 512 (5x5) configuration, where it achieved a 100% win rate. This indicates that the larger board size provides ample room for strategic moves and reduces the chances of early board saturation. In contrast, Algorithm 1 performed poorly in most configurations, with a win rate ranging from 0% to 15%. Its best performance occurred in the 256 (4x4) game, where its win rate was 15%. This suggests that Algorithm 1 performs slightly better on mid-sized boards, but still falls short compared to Algorithm 2.

Overall, the results indicate that larger boards decrease the difficulty of the reverse game and that Algorithm 2 is better suited to solve the Reverse 2048 game variations. These findings also indicate that the board size is a crucial factor in game difficulty and game performance

4.2 Test 2

The results confirm Algorithm 2's dominance in all reverse game variations and board sizes, with a few exceptions. Algorithm 2 consistently outperformed Algorithm 1 by achieving a higher win rate in almost all game configurations, thus indicating its strategic superiority.

In the Reverse 512 game, Algorithm 2 dominated the larger board sizes with a 100% win rate on the 5x5 board and a 60% win rate on the 4x4 board, while Algorithm 1 failed to win any games in these configurations. This implies that when Algorithm 2 is given adequate space to move around, it performs exceptionally well in difficult situations. The same result as Test 1 occurred for the 3x3 board, where neither Algorithm was able to solve the game, which further supports the idea that having little board space causes early saturation.

In the Reverse 256 game, Algorithm 2 performed better than Algorithm 1. It outperformed Algorithm 1, with a win rate of 90% for the 4x4 board, 80% for the 5x5 board and even a 10% on the limited 3x3 board. Algorithm 1 showed minor success on the 5x5 board with a win rate of 20%, demonstrating that it functions reasonably well when the difficulty is lowered and the board size is increased.

In the Reverse 128 game, Algorithm 1 achieved a win rate of 10% in both the 4x4 and 5x5 boards and Algorithm 2 achieved a win rate of 90% on both boards. Algorithm 2's consistent win rate as board sizes increased indicates a steady performance level in reverse games with less difficulty. For board 3x3, the win rate for Algorithm 2 was

60%, while Algorithm 1 still failed to win a game. This increase in Algorithm 2's win rate implies that a reverse game with a lower starting number is easier to solve than the 256 game and the 512 game.

4.3 Overall

All test results confirm that Algorithm 2 was more effective in nearly all Reverse Game variations. These results highlight several trade-offs encountered. While Algorithm 1 offered faster decision making and was easier to implement, it generally underperformed in complex configurations. Algorithm 2, however, demonstrated greater success by evaluating future possibilities. Furthermore, larger board sizes reduced game difficulty as both algorithms had ample space to manoeuvre tiles.

To improve the project, Algorithm 2 could be enhanced by using a deeper search instead of a 2. Exploring other complex AI algorithms like Minimax or Monte Carlo Tree Search could offer valuable solutions.

10. Conclusion

This project successfully implemented two AI algorithms, Greedy and Expectimax, to solve variations of the Reverse 2048 game. The algorithms were challenged to minimise the tile values, and through systematic testing across various board sizes and starting values, it was evident that the Expectimax algorithm (Algorithm 2) consistently outperformed the Greedy algorithm (Algorithm 1) as expected. The results emphasise the importance of strategy in solving Reverse 2048, and demonstrate how the board sizes and initial tile values influence the algorithm's performance.

References

- Bach, M. (2022, 12 15). *The future of Puzzles: How Artificial Intelligence Is Changing the Game*. Kubiya Games. <https://kubiya.com/blogs/mechanical-puzzles-blog/puzzles-and-artificial-intelligence-ai?srsId=AfmBOoq8GgvqKchsNGsoxYfDFZ-3cPjPfsQ2Td3xoLStPftSyXW1zlbq> [Accessed 03/05/2025]
- Chowdhury, G., & Dhamodaran, V. (2014). 2048 Using Expectimax. *University of Massachusetts Lowell Department of Computer Science*.
- Design Gurus Team. (2020). *What is the optimal algorithm for for 2048 game?* Design Gurus. <https://www.designgurus.io/answers/detail/what-is-the-optimal-algorithm-for-2048-game> [Accessed 04/05/2025]

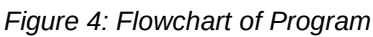


Table 3: Project Times Management

Total (Actual time)	Expected time	Total (Actual time)	Expected time
01:00:00	00:30:00	01:00:00	01:00:00
00:30:00	00:10:00	01:10:00	01:00:00
01:00:00	01:00:00	02:10:00	01:00:00
01:30:00	01:00:00	02:03:00	01:30:00
02:00:00	01:30:00	01:30:00	01:00:00
00:53:00	00:40:00	02:00:00	01:30:00
01:10:00	01:00:00	05:00:00	03:00:00
01:10:00	01:00:00	01:20:00	01:00:00
03:40:00	01:40:00	01:30:00	01:00:00
01:00:00	00:30:00	01:20:00	01:00:00
01:30:00	01:00:00	01:00:00	01:00:00
01:00:00	01:30:00	00:35:00	00:30:00
01:50:00	01:00:00	01:00:00	01:00:00
01:40:00	01:30:00	01:00:00	01:00:00
02:30:00	01:30:00	00:40:00	01:00:00
00:30:00	00:30:00	01:00:00	00:40:00
01:10:00	02:00:00	00:25:00	00:30:00
00:30:00	00:30:00	00:45:00	01:00:00
02:05:00	02:00:00	01:30:00	01:30:00
02:00:00	02:00:00	01:00:00	01:00:00
01:00:00	01:00:00	00:50:00	00:30:00
01:30:00	01:00:00	02:30:00	01:30:00
03:00:00	02:00:00	01:10:00	01:00:00
01:30:00	01:00:00	02:00:00	01:30:00
03:00:00	02:00:00	00:55:00	00:50:00
00:40:00	00:30:00	00:45:00	00:30:00
00:50:00	00:30:00	00:45:00	00:30:00
00:30:00	00:30:00	01:10:00	00:30:00
01:35:00	01:00:00	00:55:00	00:30:00

Total Time Spent:	81:11:00
Total Expected Time:	61:30:00

Discussion:

The project was planned to take 61 hours and 30 minutes, however the actual time taken for this project is 81 hours and 11 minutes. The project took 19 hours and 41 minutes more to complete. This is 30% more than anticipated. The reasons for the overrun in time are unexpected difficulty with coding, spending excessive time debugging the code, and an underestimation of the initial times. In future, a time buffer will be used to accommodate extra time for unexpected scenarios.